

**SECURITY  
ASSESSMENT**

PREPARED FOR  
**FUN TOKEN MULTICHAIN**

# Table of Contents

---

|                                    |    |
|------------------------------------|----|
| Revision History & Version Control | 03 |
| 1. Executive Summary               | 04 |
| 2. Techniques and Methods          | 08 |
| 3. Summary of Findings             | 09 |
| 4. Findings                        | 10 |
| 5. Disclaimer                      | 16 |

## Revision History & Version Control

| Version | Date           | Author's          | Description    |
|---------|----------------|-------------------|----------------|
| 1.0     | April 26, 2026 | Gurkiart, M Gowda | Initial Report |

OxTeam conducted a comprehensive Security Audit to ensure the overall code quality, security, and correctness. The review focused on ensuring that the code functions as intended, identifying potential vulnerabilities, and safeguarding the integrity of operations against possible attacks.

# 1.0 Executive Summary

## 1.1 Overview

OxTeam partnered with the Fun Token Multichain team to conduct a security audit of their smart contracts. The audit involved a manual review of the codebase to ensure the contracts are secure and reliable before deployment. This process was carried out to provide confidence to the project team and its community regarding the safety of the deployed contracts.



## 1.2 Scope

The scope of this audit involved a thorough analysis of the Fun Token Multichain Smart contracts, focusing on evaluating its quality, rigorously assessing its security, and carefully verifying the correctness of the code to ensure it functions as intended without any vulnerabilities.

Language

**SOLIDITY**

Date of Received Scope

**Apr 25, 2026**

Initial Commit

**7b359eeb57c1adfa8a858886796b7f650abd1886**

Fixed Commit

**N/A**

Repository

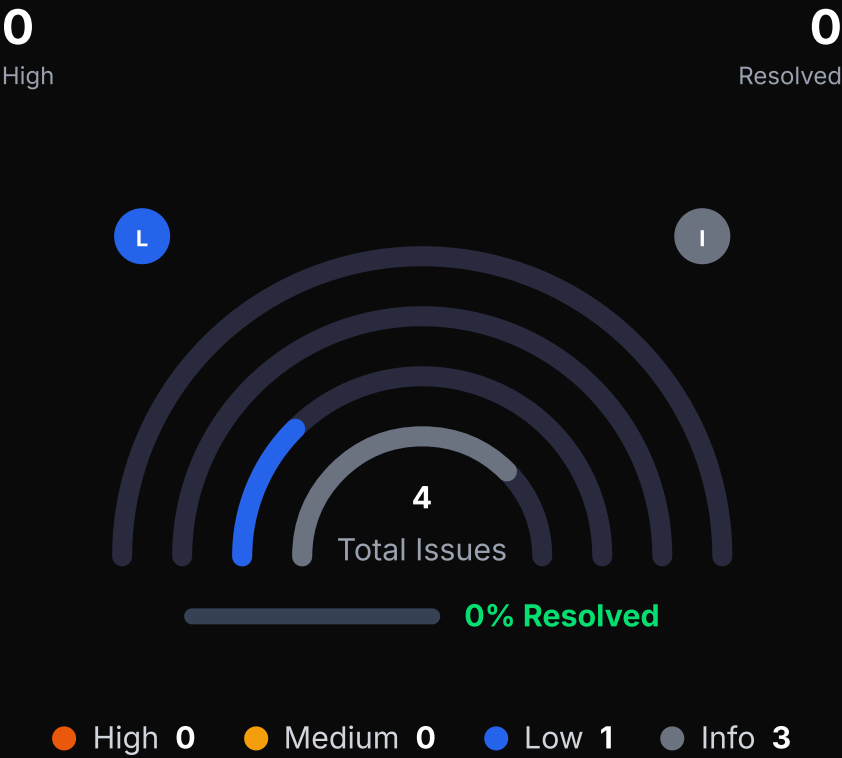
<https://github.com/shubhodeep1/fun-token-multi-chain/tree/main/contracts>

### ● In-Scope Files

- contracts/FunOFT.sol
- contracts/FunOFTAdapter.sol

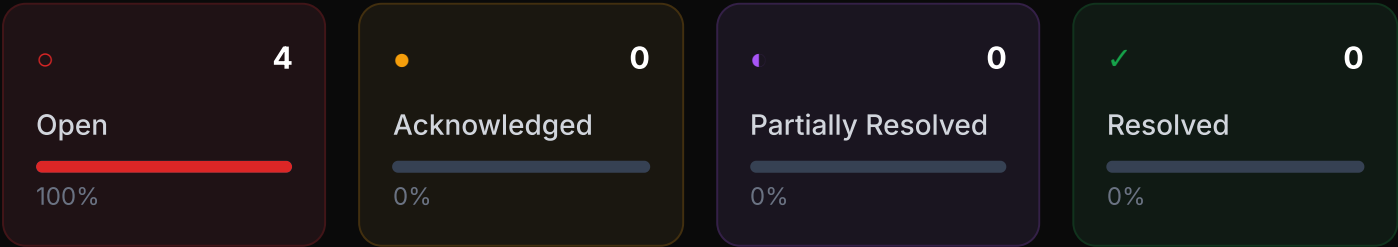
- **OUT-OF-SCOPE:** External Smart contracts code, other imported code.

1.3 Vulnerability Summary



1.4 Issue Status Summary

| Status / Severity  | High | Medium | Low | Info |
|--------------------|------|--------|-----|------|
| Open               | 0    | 0      | 1   | 3    |
| Acknowledged       | 0    | 0      | 0   | 0    |
| Partially Resolved | 0    | 0      | 0   | 0    |
| Resolved           | 0    | 0      | 0   | 0    |



## 1.5 Type of Severity

### High

Issues that pose an immediate and severe risk to the protocol's funds, user assets, or core system integrity, or significant security vulnerabilities that could lead to substantial financial loss or compromise of important protocol functionality. These issues require prompt attention and immediate remediation before deployment to prevent potential exploitation or complete loss of funds.

### Medium

Moderate security concerns that could potentially be exploited under specific conditions or may impact protocol functionality. While not immediately critical, these issues should be resolved to ensure long-term security and reliability of the system.

### Low

Minor issues that do not pose significant security risks but represent deviations from best practices or code quality standards. These findings improve code maintainability, readability, and adherence to industry conventions.

### Informational

Observations and recommendations that enhance code quality, gas efficiency, or overall design. These are non-security findings that provide suggestions for optimization, documentation improvements, or architectural considerations.

## 1.6 Type of Issue Status

### Open

Finding awaiting remediation by the development team.

### Resolved

Issue has been addressed and verified by the audit team.

### Acknowledged

Team has reviewed and accepted the risk without immediate fix.

### Partially Resolved

Mitigation applied but complete resolution pending further work.

## 2. Techniques and Methods

The following techniques, methods, and tools were used to conduct a comprehensive security review of the smart contracts.

### Structural Analysis

Examination of overall design, architecture, and code organization to ensure scalability, maintainability, and adherence to industry standards.

### Static Analysis

Automated scanning for common vulnerabilities including reentrancy, arithmetic errors, access control issues, and potential denial-of-service vectors.

### Manual Code Review

Line-by-line examination of contract logic to verify intended functionality, validate business requirements, and identify issues beyond automated detection.

### Dynamic Analysis

Execution of contracts in controlled environments with comprehensive test cases to observe behavior, verify gas efficiency, and ensure correct state transitions.

### Platforms and Tools Used to Audit

[Remix IDE](#)[Foundry](#)[Slither](#)[Hardhat](#)[Solhint](#)



### 3. Summary of Findings

| ID   | Title                                                            | Severity | Status |
|------|------------------------------------------------------------------|----------|--------|
| L-01 | Borrower Missing Zero-Address Validation in FunOFTAdapter.send() | Low      | Open   |
| I-01 | Redundant custom events for pause/unpause                        | Info     | Open   |
| I-02 | Redundant Override: _debitView Adds No Functional Value.         | Info     | Open   |
| I-03 | Inconsistent Pause Enforcement via _debitView                    | Info     | Open   |

● Resolved    ● Partially Resolved    ● Acknowledged    ● Open

## 4. Findings

### [L-1] Borrower Missing Zero-Address Validation in FunOFTAdapter.send()

LOW

Open

#### CONTRACT

FunOFTAdapter

#### LOCATION

FunOFTAdapter.send()

#### Description

FunOFTAdapter.send() does not validate that *sendParam.to*  $\neq$  *bytes32(0)* and *refundAddress*. A user who accidentally (or maliciously) passes *address(0)* as the recipient will have their tokens permanently burned with no revert, no warning, and no recovery path.

The call chain *send()*  $\rightarrow$  ...  $\rightarrow$  *credit()* has no zero-address check at the entry point. The LayerZero *OFT.\_credit()* function silently redirects *address(0)* to *address(0xdead)*:

```
if (_to == address(0x0)) to = address(0xdead); // mint(...) does not support address(0x0)
```

This means no revert occurs — tokens are simply minted to an uncontrolled burn address.

#### Impact

When *send()* is called with *to* = *bytes32(0)*:

1. **Source chain:** Sender's tokens are debited and **locked in the adapter** (or burned from the OFT)
2. **Destination chain:** Tokens are minted to *0x000...dEaD*, an uncontrolled address
3. **Result:** Permanent, irrecoverable loss of funds

In the below POC we have passed zero address in *sendParam* and *refundAddress* and function is not able to detect address zero and loss of funds takes place and funds are in dead address.

$\})$

```

gurkiratsingh@Gurkirats-MacBook-Air fun-token-multi-chain-main % npm run test
> fun-token-multi-chain@0.1.0 test
> hardhat test

CrossChain Integration

  -- BEFORE send() --
  Sender source token balance : 1000000000000
  Adapter locked source tokens : 0
  Dest OFT at address(0)      : 0
  Dest OFT at 0xdead          : 0
  Dest OFT at user            : 0

  -- AFTER send() --
  Sender source token balance : 999900000000
  Adapter locked source tokens : 100000000
  Dest OFT at address(0)      : 0
  Dest OFT at 0xdead          : 100000000
  Dest OFT at user            : 0

  -- SUMMARY --
  Tokens debited from sender   : 100000000
  Tokens locked in adapter     : 100000000
  Tokens minted to address(0)  : 0
  Tokens minted to 0xdead      : 100000000
  Tokens minted to user        : 0

  ✓ sends with to=address(0) - traces where tokens actually end up (211ms)

1 passing (483ms)

```

## Recommendation

Add a zero address check at the top of FunOFTAdapter.send() :

```

function send(SendParam calldata _sendParam, MessagingFee calldata _fee, address _refundAddress)
    external
    payable
    override
    nonReentrant
    whenNotPaused
    returns (MessagingReceipt memory msgReceipt, OFTReceipt memory oftReceipt)
{
    if (_sendParam.to == bytes32(0)) {
        revert AddressZeroRecipient();
    }
    if (_refundAddress == address(0)) {
        revert AddressZeroRefundAddress();
    }
    return _send(_sendParam, _fee, _refundAddress);
}

```

**[I-1] Redundant custom events for pause/unpause**

INFO

Open

**CONTRACT**

FunOFT, FunOFTAdapter

**LOCATION**

pause, unpause

**Description**

The contracts define and emit custom pause/unpause events (OftPaused, OftUnpaused, AdapterPaused, AdapterUnpaused) despite inheriting from OpenZeppelin's Pausable, which already emits Paused(address) and Unpaused(address) events.

**Impact**

This results in **duplicate event emissions** for the same state change, causing unnecessary gas consumption.

**Recommendation**

Remove custom events.

**[I-2] Redundant Override: \_debitView Adds No Functional Value.**

INFO

Open

**CONTRACT**

FunOFT

**LOCATION**

\_debitView

**Description**

The function *debitView* in *FunOFT* overrides the parent implementation but does not introduce any new logic. It simply forwards the call to the parent contract using *super.debitView*.

**Impact**

Slightly increases deployment cost and execution overhead.

**Recommendation**

If no custom logic is required, delete the function entirely:

The contract will automatically inherit the correct behavior from the parent (OFT).

If the override was intentional, implement meaningful logic.

**[I-3] Inconsistent Pause Enforcement via \_debitView**

INFO

Open

**CONTRACT**

FunOFT, FunOFTAdapter

**LOCATION**

\_debitView)

**Description**

The contract implements a pause mechanism using Pausable, restricting token bridging via *debit when paused*. However, the corresponding view function `debitView` does **not enforce the paused state**, leading to inconsistent behavior between execution and simulation.

This allows transfer simulations (quotes) to succeed even when actual transfers are blocked.

**Impact**

- `_debitView` returns valid results while paused
- `_debit` reverts
- Leads to incorrect assumptions by users and integrators

**Recommendation**

- Option 1 is to enforce Pause in `_debitView`
- Option 2 Explicitly document that:  
"Quote/simulation functions remain active during pause" & Ensure integrators do not rely on them for execution guarantees

## Disclaimer

The OxTeam smart contract audit is not a security warranty, investment advice, or an endorsement of the Fun Token. This audit does not guarantee that the code is completely free of vulnerabilities, as security risks can evolve over time. Users and investors should conduct their own due diligence before interacting with the Token. OxTeam is not responsible for any financial losses, hacks, or exploits that may occur after the audit. The findings in this report reflect the security state of the code at the time of assessment. By using this report, you acknowledge that smart contract security is an ongoing process and that OxTeam shall not be held liable for any direct or indirect damages resulting from the use of the audited code.



## About OxTeam

OxTeam is a Web3-focused cybersecurity company dedicated to strengthening blockchain systems and decentralized applications. We help teams adopt blockchain technology with confidence by ensuring their smart contracts and infrastructure are secure, reliable, and production-ready.

Security is at the core of everything we do. Our services are designed to support smooth adoption across the Web3 ecosystem, enabling projects to scale safely while minimizing technical and financial risk.

OxTeam continues to grow its partnerships with developers, startups, and organizations across Web3, working together to build secure, resilient, and future-ready decentralized products.

Website: <https://Oxteam.space>

Contact: [info@Oxteam.space](mailto:info@Oxteam.space)

